CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G5
System Integration and Architecture

System Design Document (SDD)

Project Title: BayanLink: Barangay Services Portal

Prepared By: Ceniza, Jian Marc

Version: 1.0

Date: February 20, 20265
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	02/20/2026	Ceniza, Jian Marc	Initial draft	Draft
0.2	Date	Ceniza, Jian Marc	Added API specifications	Review
0.3	Date	Ceniza, Jian Marc	Updated database design	Review
0.4	Date	Ceniza, Jian Marc	Added UI/UX designs	Review
0.5	Date	Ceniza, Jian Marc	Incorporated feedback	Revised
0.6	02/27/2026	Ceniza, Jian Marc	Final review and corrections	Final Review
1.0	03/06/2026	Ceniza, Jian Marc	Completion of all required content.	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4

1.0 INTRODUCTION 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5

3.0 NON-FUNCTIONAL REQUIREMENTS 8

4.0 SYSTEM ARCHITECTURE 9

5.0 API CONTRACT & COMMUNICATION 10

Authentication Endpoints 11

User Registration 11

User Login 11

6.0 DATABASE DESIGN 13

7.0 UI/UX DESIGN 14

8.0 PLAN 17

EXECUTIVE SUMMARY

1.1 Project Overview

BayanLink is a centralized digital platform designed for residents and officials of a Barangay to manage administrative actions and health-related services. Acting as a digital bridge between the community and the local government unit (LGU), it replaces manual, paper-based workflows with automated processes for official document issuance and health consultation scheduling. The system utilizes a Spring Boot backend integrated with an online PostgreSQL database hosted via Supabase to provide a production-ready, scalable environment for resident data.

1.2 Objectives

1. Develop a digital portal to facilitate user registration, secure login, and profile management for community members.

2. Implement a transactional system for requesting official barangay documents such as Barangay Clearances and Certificates of Indigency.

3. Create a Community Scheduling Module for booking medical appointments at the local health center and counseling sessions with the Barangay Captain.

4. Establish Role-Based Access Control (RBAC) to provide specific dashboards and functionalities for residents, administrative staff, and health workers.

5. Modernize the community's database using cloud-hosted relational storage to ensure records are accurate and accessible on a larger scale.

1.3 Scope

Included Features:

- **User Authentication and Access Control:** Multi-role registration and secure unified login using BCrypt-encrypted passwords.
- **User Profile Management:** A "Digital ID" storing personal information, verification status, and recent activity logs.
- **Integrated E-Service Dashboard:** A real-time announcement feed with quick-action navigation for residents and monitoring widgets for staff.
- **Digital Document Transaction Module:** Resident forms for document requests with file-upload for IDs and a management dashboard for staff to track progress.
- **Community Scheduling Module:** An interactive calendar for residents to browse available slots and a master schedule view for health workers.

- **Digital Storage Infrastructure:** Using PostgreSQL on Supabase for data persistence and remote accessibility across the website and mobile.

Excluded Features:

- **Financial Payments**: The system will not handle digital transactions; all payments remain face-to-face.
- **Real-time Medical Diagnosis**: The portal focuses strictly on scheduling health services rather than providing remote medical consultations.
- **Cross-Barangay Functions**: The system is limited to the administrative functions and records of a single barangay.
- **Inventory Automation**: While managing clinic capacity, the scope does not include automated medical inventory or supply chain management.

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the BayanLink: Barangay Services Portal. It provides detailed requirements, architectural decisions, API contracts, database design, and implementation roadmap to guide development and ensure all components integrate seamlessly. The purpose of this system is to provide a centralized digital platform for residents and officials of a Barangay to manage administrative actions and health-related services. It aims to replace manual, paper-based workflows with automated processes, specifically for official document issuing and health consultation scheduling.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: BayanLink: Barangay Services Portal
Domain: Local Government (Barangay) & Community Health
Primary Users:

1. **Residents**: General community members seeking documents and health services.
2. **Barangay Staff/Officials**: Administrative personnel managing requests and announcements.

3. **Health Workers**: Medical staff managing clinic schedules and patient records.

4. **System Admin**: Technical personnel managing security and user roles.

Problem Statement: Community administrative and health-related workflows are currently manual, paper-based, or rely on verbal announcements, leading to inaccuracies and updated resident data.
Solution: A centralized digital bridge (portal) that automates official document issuance and health consultation scheduling while providing a digital database for resident status updates.

2.2 Core User Journeys

Journey 1: Resident Document Request

1. The resident visits the digital platform to begin the onboarding process.
2. The user enters their full name, email address, and birthdate into the registration form.
3. The resident creates a password, which the system hashes using the BCrypt function before storing it in the PostgreSQL database on Supabase.
4. To satisfy registration requirements, the resident must provide a valid government-issued ID for account verification.
5. Upon successful account creation, the resident authenticates via the secure login interface and is redirected to their specific home dashboard.

Journey 2: Resident Document Request

1. The resident logs in with their registered email/username and secure credentials.
2. From the main dashboard, the resident selects "Quick Action" buttons to access the Digital Document Transaction Module.
3. The resident selects a document type, such as a Barangay Clearance or Certificate of Indigency, and completes the required request form.
4. The resident utilizes the file-upload section to attach digital copies of their identification.
5. After submission, the resident monitors the progress of their application through a "Request History" table.

Journey 3: Resident Health Center & Counseling Appointment

1. The resident accesses the system using their secure account.

2. The resident navigates to the Community Scheduling Module to browse an interactive calendar of available time slots.
3. The user chooses a slot for health services or counseling with the Barangay Captain.
4. Upon selection, the system generates an appointment confirmation and a unique reference number.
5. The resident presents the reference number at the barangay health center or hall during their scheduled visit.

Journey 4: Administrative Management (Staff & Health Workers)

1. Staff and health workers log in; the system identifies their role and redirects them to their specialized administrative panels.
2. Barangay officials view a centralized dashboard of incoming requests, verify uploaded documents, and update statuses to "In Progress" or "Ready for Pickup".
3. Nurses and RHU staff access a master schedule to view the list of confirmed patients for the day.
4. Health workers manage clinic capacity by blocking off holidays and marking patients as "Attended" or "Cancelled" to maintain accurate records.

2.3 Feature List (MoSCoW)

MUST HAVE

1. **User Authentication**: Multi-role registration, secure login, and session management using Spring Security.
2. **Document Transactions**: Digital application forms, file uploads for IDs, and status tracking.
3. **Health Scheduling**: Interactive calendar for residents and clinic capacity management for health workers.
4. **Role-Based Access Control (RBAC)**: Unique dashboards and restricted data access based on user type.

SHOULD HAVE

1. **User Profile Management**: "Digital ID" view and the ability to update contact details and profile pictures.
2. **Announcement Feed**: Real-time community notices and health alerts on the main dashboard.
3. **Audit Logs**: Activity logs on the profile page showing recent system interactions.

COULD HAVE

1. **National ID Integration**: Potential future integration with national databases.
2. **Municipal Sync**: Integration with higher-level municipal health systems.
3. **Advanced Indexing**: Database query optimization for high-volume record retrieval.

WON'T HAVE

1. **Digital Payments**: Financial transactions remain face-to-face.
2. **Remote Diagnosis**: The system does not provide real-time medical diagnosis.
3. **Cross-Barangay Management**: Limited to the administrative functions of a single barangay.

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens**: Registration, Login, Forgot Password.
- **Fields**: Email, Password, Full Name, Birthdate.
- **Validation**: Email format, password strength, and uniqueness checks.
- **Security**: JWT/Auth Tokens, Spring Security session management, and password hashing with BCrypt.
- **API Endpoints**:
    - `POST /auth/register`
    - `POST /auth/login`
    - `POST /auth/logout`

Feature: User Profile Management

- **Screens**: User Profile, Edit Profile.
- **Fields**: Full Name, Email Address, Birthdate, Age, Contact Number, Profile Picture.
- **Security**: Current password verification required for credential changes.
- **API Endpoints**:
  - `GET /profile/{userId}`
  - `PUT /profile/{userId}`
  - `DELETE /profile/{userId}`

Feature: Digital Document Module

- **Screens**: Request Page (Resident), Management Page (Staff).
- **Display**: List of document types, request history table, and urgency sorting.
- **Functions**: File upload for identification and status updates (e.g., "Ready for Pickup").
- **Persistence**: PostgreSQL (Supabase) via Spring Data JPA.
- **API Endpoints**:
  - `GET /documents/requests`
  - `POST /documents/requests`
  - `PUT /documents/requests/{requestId}`
  - `DELETE /documents/requests/{requestId}`

Feature: Community Scheduling

- **Screens**: Scheduling Calendar (Resident), Monitoring Page (Health Worker).
- **Functions**: Time slot selection, reference number generation, and slot blocking.
- **Data Collected**: Appointment date/time, service type, and resident reference.
- **API Endpoints**:
  - `GET /appointments`
  - `POST /appointments`
  - `PUT /appointments/{appointmentId}`
  - `DELETE /appointments/{appointmentId}`

2.5 Acceptance Criteria

AC-1: Successful User Registration and Role Assignment

- Given I am a new resident without an existing account.
- When I enter a valid email, full name, birthdate, and a strong password.
- And I upload a valid government-issued ID for account verification.
- Then the system should hash my password using BCrypt and store my record in the Supabase PostgreSQL database.
- And I should be automatically authenticated and redirected to the Resident Dashboard.

AC-2: Resident Digital Document Request Flow

- Given I am a logged-in resident on the dashboard.
- When I select a document type such as "Barangay Clearance" and fill out the request form.
- And I successfully uploaded the required digital identification files.
- And I submit the request via the POST `/documents/requests` endpoint.
- Then the request should appear in my Request History table with a "Pending" status.
- And a notification or widget update should appear on the Staff Dashboard indicating a new pending request.

AC-3: Health Center Appointment Booking

- Given I am an authenticated resident accessing the Community Scheduling Module.
- When I select an available time slot from the interactive calendar for a health service.
- And I confirm the booking via the POST `/appointments` endpoint.
- Then the system should generate a unique appointment reference number for me to save.
- And the selected time slot should be blocked from further bookings to prevent overcapacity.

AC-4: Administrative Management and Status Updates

- Given I am logged in as a member of the Barangay Staff.
- When I review a resident's document request and verify their uploaded ID.
- And I update the status to "Ready for Pickup" via the PUT `/documents/requests/{requestId}` endpoint.
- Then the resident's Request History table must reflect the updated status in real-time.

AC-5: Clinic Attendance and Record Keeping

- Given I am logged in as a Health Worker viewing the master schedule.
- When a resident arrives for their scheduled visit, I mark them as "Attended".
- And I save the update via the PUT `/appointments/{appointmentId}` endpoint.
- Then the system must persist this status in the database to maintain accurate community health records.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- Response Latency: The system shall load the Main Dashboard and the Universal Profile page within 5 seconds under standard local network conditions.
- Concurrent Capacity: The Spring Boot backend shall be configured to handle at least 100 concurrent users without measurable degradation in transaction processing speed.
- Database Optimization: All search queries for document requests and health appointments must be indexed in the PostgreSQL database to ensure quick retrieval as records grow.
- Persistence Integrity: All transactions must follow ACID properties provided by Spring Data JPA to ensure data is never lost during system failures.

3.2 Security Requirements

- Data Encryption: All user passwords must be hashed using the BCrypt password-hashing function before being stored in the Supabase database.

- Authorization (RBAC): The system shall enforce Role-Based Access Control, ensuring only users with "Admin" or "Health Worker" roles can access transactional records that are not their own.
- Session Integrity: Secure session management shall be implemented via Spring Security to prevent unauthorized access through session hijacking.
- Privacy Compliance: All personal and medical data must be handled in accordance with local data privacy standards to prevent exposure to unauthorized users.
- Credential Verification: The system shall require users to input their current password as a verification step before allowing changes to sensitive profile data.

3.3 Compatibility Requirements

- Browser Support: The web portal shall support Google Chrome, Mozilla Firefox, and Microsoft Edge.
- Operating Environment: The client-side must be accessible on any device with a modern web browser, including smartphones, tablets, and PCs.
- Responsive Interface: The UI shall be optimized for both desktop and mobile web browsers to accommodate residents who primarily access the portal via smartphones.
- Development Compatibility: The system is built to run on Java Runtime Environment (JRE) 17.

3.4 Usability Requirements

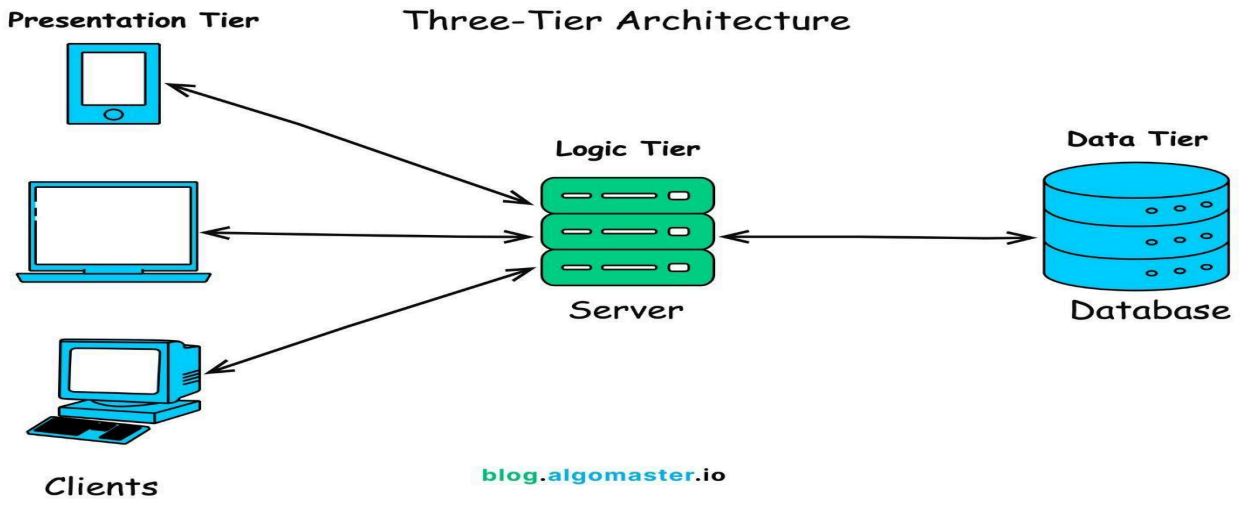
- Simplified Navigation: Transitioning between the Auth, Profile, and Transactional modules shall require no more than two clicks from the dashboard.
- User Guidance: The system shall provide clear error messages and "Success" notifications (e.g., "Appointment Booked Successfully") to guide users through processes.
- System Availability: The application shall target 99% availability during the barangay's official operating hours, such as 8:00 AM to 5:00 PM.

- Fault Tolerance: The system shall implement custom error pages (e.g., 404 or 500) to prevent exposing raw stack traces to the user.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, XML-Based
- **Build Tools:** Maven (Backend), npm (Web), Gradle (Android)
- **Deployment:** Render(Backend), Render(Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	http://localhost:8080/api/users/login
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header

Response Structure	{
	"success": true,
	"data": {
	"token":
	"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
	",

	"userEmail": "hamex@company.com",
	"role": "admin"
	},
	"error": null,
	"timestamp": "2026-02-27T09:40:50Z"
	}
	
	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH_001",
	"message": "Invalid credentials",
	"details": {
	"reason": "Email or password does not match our records."
	}
	},
	"timestamp": "2026-02-27T09:40:55Z"
	}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/api/users/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"userEmail": "johndoe@email.com",
	"userPassword": "securePassword123",
	"confirmPassword": "securePassword123"
	}

Response Structure	{
	"success": true,
	"data": {
	"userId": 102,
	"userEmail": "johndoe@email.com",
	"message": "Registration Successful!"
	},

	"error": null,
	"timestamp": "2026-02-27T10:15:30Z"
	}
	{
	"success": false,
	"data": null,
	"error": {
	"code": "REG_VALIDATION_ERROR",
	"message": "Registration failed",
	"details": {
	"reason": "Passwords do not match."
	}
	},
	"timestamp": "2026-02-27T10:15:35Z"
	}

User Login

Description	User Login
API URL	/api/users/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"userEmail": "hamex@company.com",
	"userPassword": "securePassword123"
	}

Response Structure	{
	"success": true,
	"data": {
	"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
	"userEmail": "hamex@company.com",
	"role": "admin"
	},
	"error": null,
	"timestamp": "2026-02-27T09:40:50Z"
	}
	
	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH_INVALID_CREDENTIALS",
	"message": "Invalid credentials",

	"details": {
	"reason": "The email or password provided does not match our records."
	}
	},
	"timestamp": "2026-02-27T09:40:55Z"
	}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Authentication Failure	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH-001",
	"message": "Invalid credentials",
	"details": "Email or password is incorrect"
	},
	"timestamp": "2026-02-27T10:30:00Z"
	}

Registration Validation Failure	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"userEmail": "Email is required",
	"userPassword": "Must be at least 8 characters"
	}

	}, "timestamp": "2026-02-27T10:35:00Z" }

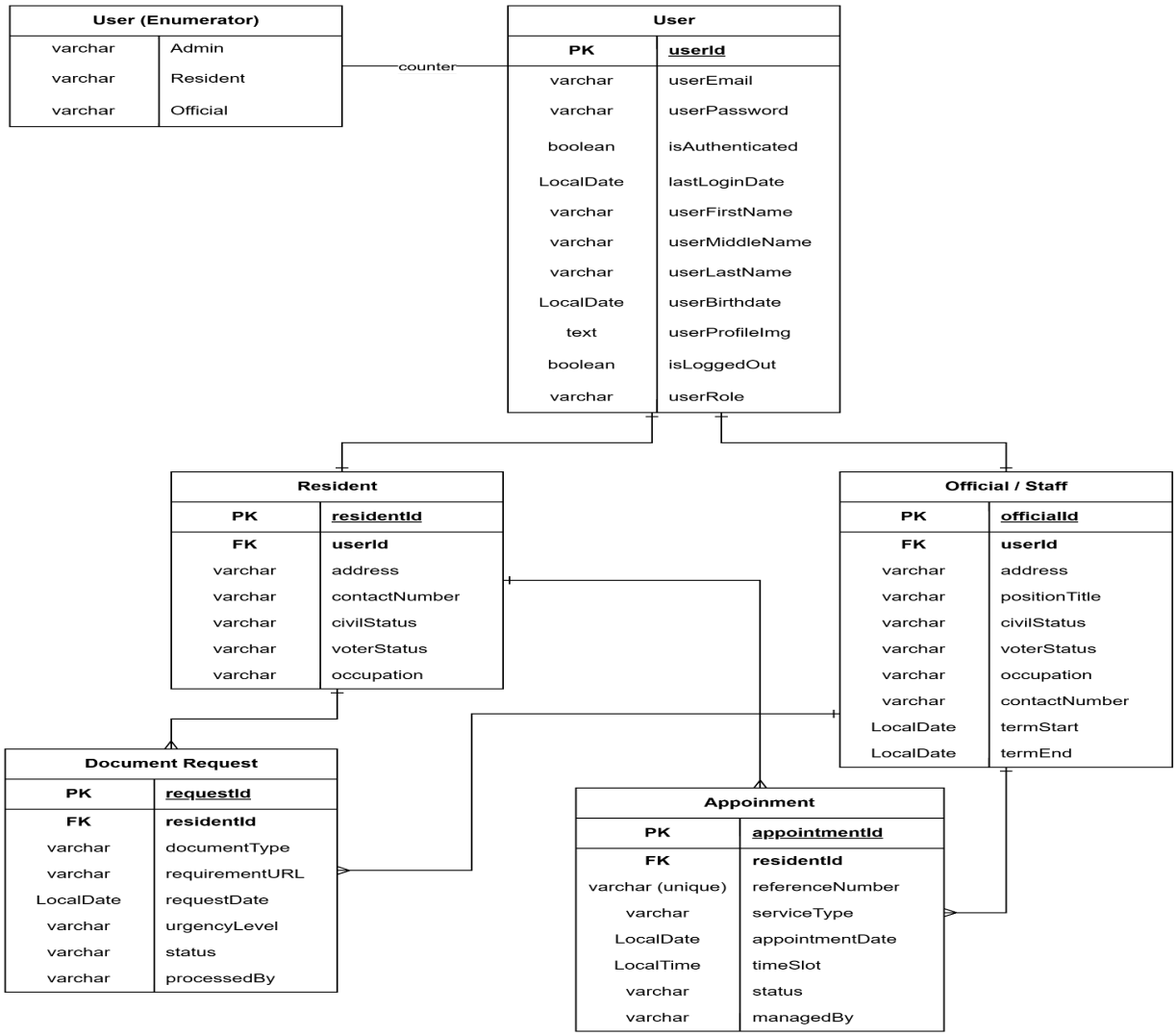
Common Error Codes

- **AUTH-001: Invalid Credentials** – Triggered during the secure unified login when the email or username does not match the BCrypt-encrypted password.
- **AUTH-002: Token Expired** – Occurs when the Spring Security session or JWT token has timed out, requiring the resident or official to log in again.
- **AUTH-003: Insufficient Permissions** – Appears when a Resident attempts to access Staff/Official management panels or when a user tries to access transactional records that are not their own.
- **VALID-001: Validation Failed** – Occurs if registration or request forms are incomplete, such as missing the required identification files in the document module.
- **DB-001: Resource Not Found** – Triggered if a specific document request or appointment reference number cannot be located in the Supabase PostgreSQL database.
- **DB-002: Duplicate Entry** – Appears if a user attempts to register with an email already present in the system or if a resident tries to book a slot that has been marked as full or blocked.
- **BUS-001: Capacity Reached** – Specific to the Community Scheduling Module when all available time slots for health services or counseling are occupied.
- **BUS-002: Verification Required** – Triggered if a resident attempts to request a document while their profile Verification Status is still "Pending" or "Unverified".
- **BUS-003: Profile Conflict** – Occurs if a user attempts to update sensitive profile data without providing their current password as a verification step.
- **SYS-001: Internal Server Error** – A general error for unexpected backend failures; the system should display a custom 500 error page instead of raw stack traces.
- **SYS-002: Database Unavailable** – Triggered if the Supabase connection is interrupted, preventing the Spring Data JPA from persisting or retrieving records.
- **SYS-003: Service Latency** – Appears if the system exceeds the target 5-second response time for dashboard or profile loading.

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)



Detailed Relationships:

- **User ↔ Resident**: Linked by a **One-to-One** relationship, where a core authentication account is extended into a community member profile.
- **User ↔ Official / Staff**: Linked by a **One-to-One** relationship, where a core account is assigned administrative or health worker privileges.
- **Resident → Document Request**: Connected via a **One-to-Many** relationship, allowing a community member to file multiple certification requests over time.
- **Resident → Appointment**: Connected via a **One-to-Many** relationship, enabling a member to schedule multiple visits for health or counseling services.

- **Official / Staff** ↔ **Document Request**: Linked by a **Many-to-One** relationship, where a single administrator manages and verifies multiple incoming requests.
- **Official / Staff** ↔ **Appointment**: Linked by a **Many-to-One** relationship, where a specific health worker or official is assigned to manage a set of scheduled slots.

Key Tables:

1. **User**: Stores primary authentication data, including **BCrypt-encrypted** passwords and JWT session status.
2. **Resident**: Contains demographic data such as address and voter status required for document verification.
3. **Official / Staff**: Stores professional details, including position titles and term duration for administrative tracking.
4. **Document Request**: Acts as the transactional log for barangay certifications, tracking urgency and processing status.
5. **Appointment**: Manages scheduling logistics, including reference numbers and specific service time slots.

Table Structure Summary:

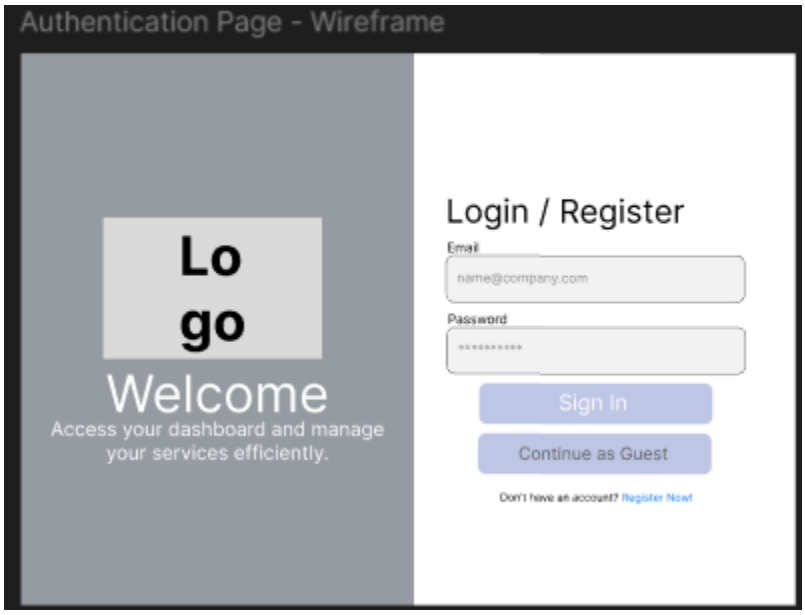
- **User**: <u>userId</u> (PK), userEmail, userPassword, isAuthenticated, lastLoginDate, userFirstName, userMiddleName, userLastName, userBirthdate, userProfileImg, isLoggedOut, userRole"
- **Resident**: <u>residentId</u> (PK), userId (FK), address, contactNumber, civilStatus, voterStatus, occupation"
- **Official / Staff**: "<u>officialId</u> (PK), userId (FK), address, positionTitle, civilStatus, voterStatus, occupation, contactNumber, termStart, termEnd"
- **Document Request**: "<u>requestId</u> (PK), residentId (FK), documentType, requirementURL, requestDate, urgencyLevel, status, processedBy (FK)"
- **Appointment**: "<u>appointmentId</u> (PK), residentId (FK), referenceNumber (unique), serviceType, appointmentDate, timeSlot, status, managedBy (FK)"

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

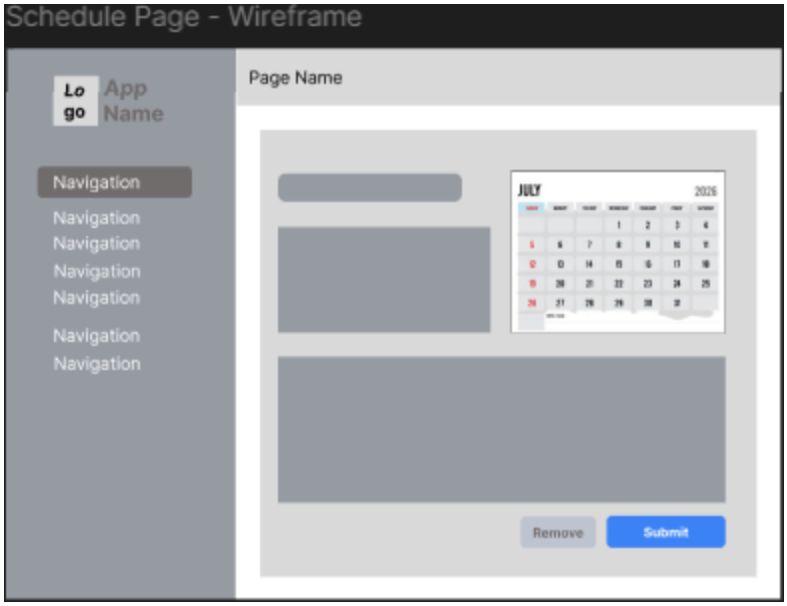
Link:
https://www.figma.com/design/0TDQBfSJu9DuC90wOV3gvM/BayanLink--Barangay-Services-Portal?node-id=0-1

Note: This should be wireframes from Figma



Authentication Page

- Split Layout: A two-tone interface with a brand/welcome section on the left and form entry on the right.
- Left Panel: Features the Logo, a large "Welcome" header, and a sub-text: "Access your dashboard and manage your services efficiently."
- Right Panel:
    - Header: "Login / Register".
    - Fields: `Email` and `Password` input boxes with placeholder text.
    - Actions: Primary `Sign In` button and a secondary `Continue as Guest` button.
    - Footer: "Don't have an account? Register Now!" link.

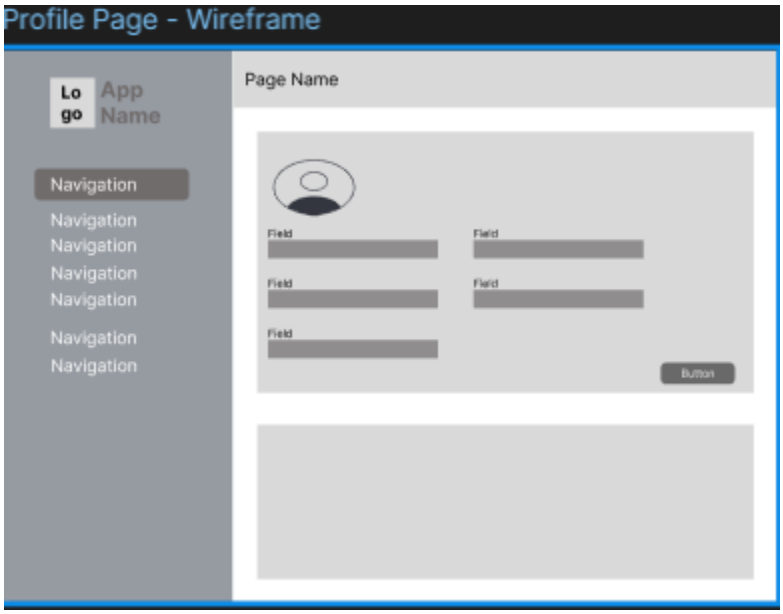


Schedule Page

- Interactive Interface:
    - Selection Group: Dropdown or selection boxes for picking a specific service or official (e.g., Barangay Captain counseling).
    - Calendar Widget: A "July 2026" interactive calendar for browsing available time slots.
- Booking Summary: A large text container to review the selected slot details before finalizing.
- Final Actions: `Remove` and a blue primary `Submit` button to generate an appointment confirmation.

Clinic Page (Health Services)

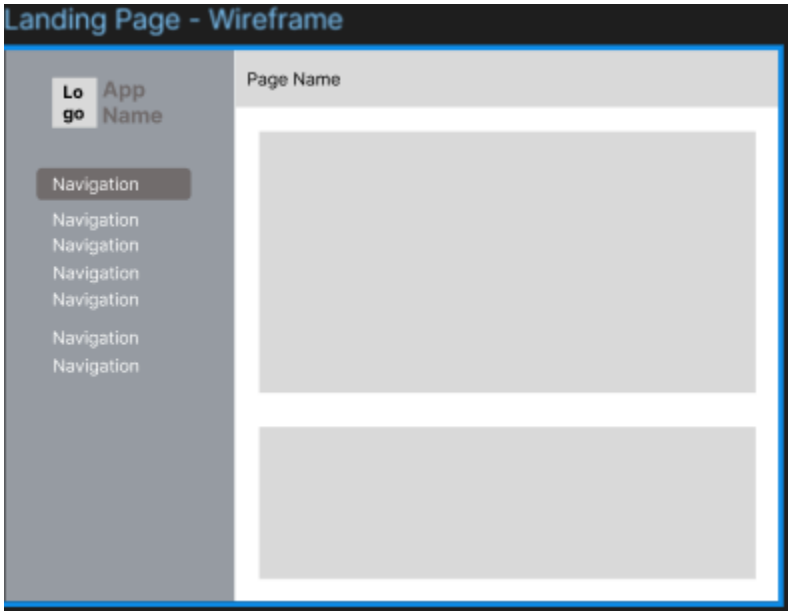
- Provider Grid: A top section featuring a 3-column grid of provider/service profiles (e.g., Nurses, Midwives, or RHU staff).
- Information Area: A large lower container for detailed service descriptions, health alerts, or clinic hours.



Profile Page

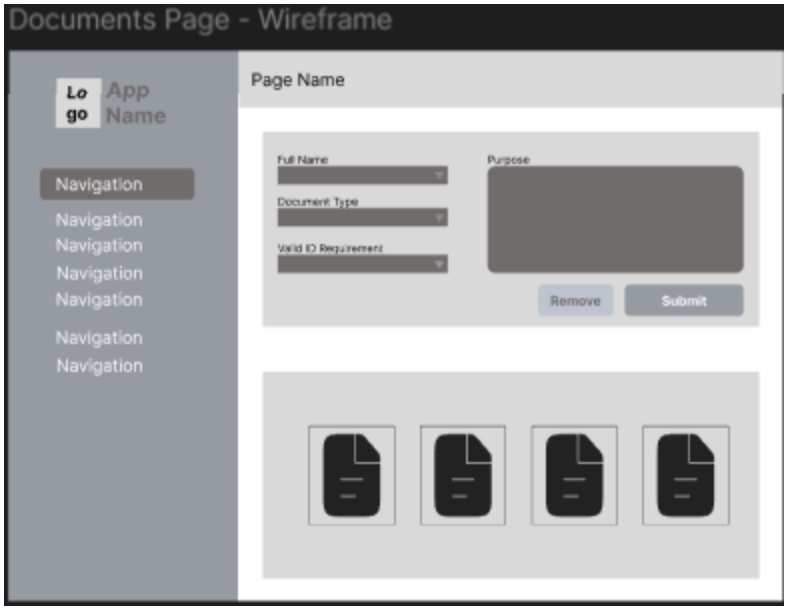
- Profile Header: A circular `Avatar/Profile Picture` placeholder.

- Data Fields: A 2x2 grid for information entry (e.g., Full Name, Email, Birthdate, Contact Number) plus a long-form address field.
- Actions: An `Edit/Save` button located at the bottom-right of the field group.
- Recent Activity: A dedicated lower container to display the "Recent Activity" logs mentioned in the FRS.



Landing Page (Service Dashboard)

- Sidebar Navigation: Logo, App Name, and a vertical list of navigation links (Dashboard, Profile, Documents, etc.).
- Top Bar: Displays the "Page Name" (e.g., Home Dashboard).
- Content Areas:
    - Main Hero/Announcement Section: A large top container for community alerts or featured services.
    - Secondary Content Section: A smaller bottom container for recent updates or quick-action status summaries.



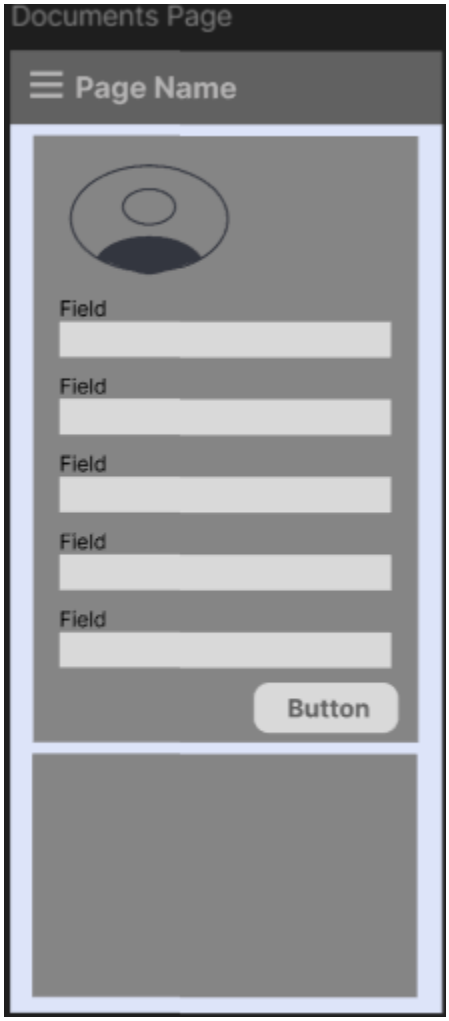
Documents Page

- Request Form:
  - Left Column: Dropdown menus for `Full Name`, `Document Type`, and `Valid ID Requirement`.
  - Right Column: A large `Purpose` text area for residents to specify request details.
  - Form Actions: `Remove` and `Submit` buttons.
- Request History: A lower container featuring a grid of document icons representing the "Request History" table for real-time tracking.

7.2 Mobile Application Wireframes

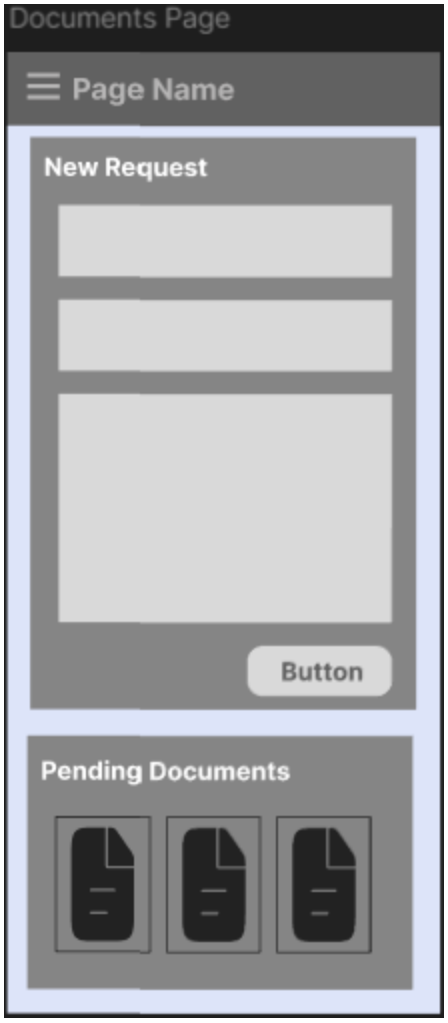
Link:
https://www.figma.com/design/0TDQBfSJu9DuC90wOV3gvM/BayanLink--Barangay-Services-Portal?node-id=7-289

Note: This should be wireframes from Figma



Mobile Profile Page

- User Header: A centered circular Avatar placeholder at the top of the main content area.
- Information Fields: A single-column vertical stack of five input fields (e.g., Full Name, Email, Birthdate, Contact, Address) to accommodate narrower screens.
- Action Button: A prominent, rounded Button at the bottom of the field list for saving changes.
- Activity Log: A large bottom container reserved for displaying "Recent Activity" logs.



Mobile Documents Page

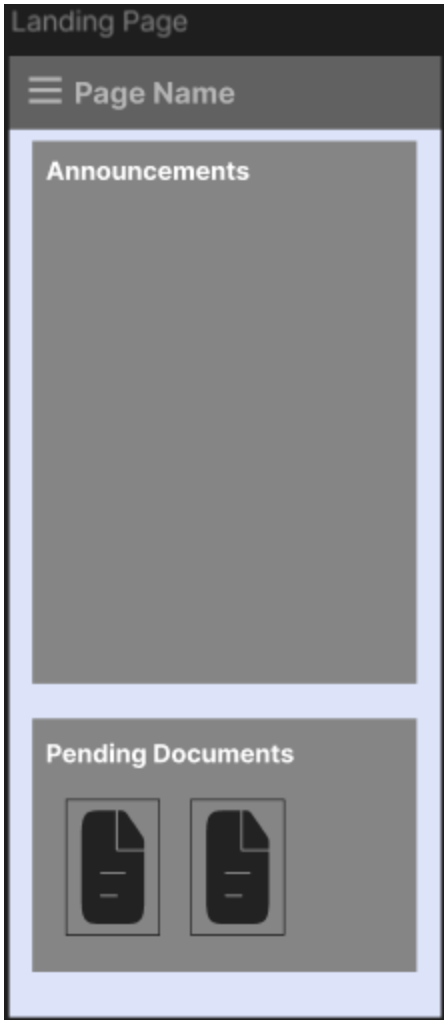
- New Request Section:
  - Vertical stack of three selection/input boxes (e.g., Document Type, Purpose, ID Upload).
  - A primary Button for submitting the request.

Mobile Clinic Page

- Staff Present Section: A top container displaying the list or grid of available health workers (nurses, midwives, or RHU staff).
- Services Available Section: A large bottom container detailing current clinic offerings and health alerts.

Mobile Schedule Page

- New Schedule Section:
    - A top selection box for choosing a service or official (e.g., Barangay Captain).
    - Interactive Calendar: A centered "July 2026" calendar widget for selecting available dates.
- Selection Summary: A large grey container below the calendar to review booking details.
- Action Button: A primary Button at the bottom to finalize the appointment and generate a reference number.



Mobile Landing Page (Dashboard)

- Announcements Section: A large, primary grey container at the top for real-time community notices and health alerts.
- Pending Documents Widget: A secondary container below the announcements featuring a horizontal or grid layout of document icons to track active requests.

Mobile Navigation & Header

- Header Bar: A dark grey top bar containing a "hamburger" menu icon and the "Page Name" centered or left-aligned.
- Navigation Drawer: A full-height vertical overlay containing the Logo, App Name, and a list of primary navigation links (e.g., Dashboard, Documents, Clinic, Schedule, Profile, Logout).

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1–2)

Week 1: Requirements & Architecture

- Day 1–2: Initial project setup and documentation of the Functional Requirements Specification.
- Day 3–4: Completion of the FRS and Non-Functional Requirements.
- Day 5–7: Finalization of the system architecture design.

Week 2: Detailed Design

- Day 1–2: API specification for authentication and transaction modules.
- Day 3–4: Relational database design for PostgreSQL on Supabase.
- Day 5–6: Creation of UI/UX wireframes for mobile and web platforms.
- Day 7: Implementation plan finalization.

Phase 2: Backend Development (Week 3–4)

Week 3: Foundation

- Day 1: Spring Boot setup with necessary dependencies like Lombok and Maven.
- Day 2: Database configuration using Spring Data JPA and defining user entities.
- Day 3: Implementation of JWT authentication and BCrypt password hashing.
- Day 4: Development of user management and profile service endpoints.
- Day 5: Initial setup of the Digital Document Transaction Module logic.

Week 4: Core Features

- Day 1: Development of the document request submission and file-upload logic.
- Day 2: Implementation of the Community Scheduling Module for health appointments.
- Day 3: Search and filtering for document requests and clinic schedules.
- Day 4: Implementation of Role-Based Access Control (RBAC) and error handling.
- Day 5: API documentation using Swagger/OpenAPI and integration testing.

Phase 3: Web Application (Week 5–6)

Week 5: Frontend Foundation

- Day 1: React setup to create the stand-alone web portal.
- Day 2: Implementation of secure registration and login pages.
- Day 3: Development of the Integrated E-Service Dashboard and announcement feed.
- Day 4: Building the Digital ID profile page.
- Day 5: Development of the resident document request forms.

Week 6: Complete Web Features

- Day 1: Implementation of the interactive clinic scheduling calendar.
- Day 2: Creation of the Request History and appointment confirmation views.
- Day 3: Building the Staff/Official Management and Monitoring dashboards.
- Day 4: Responsive design polish for mobile browser compatibility.
- Day 5: Full frontend-to-backend integration and usability testing.

Phase 4: Mobile Application (Week 7–8)

Week 7: Android Foundation

- Day 1: Android Studio project setup for the mobile client.
- Day 2: Development of mobile-optimized authentication screens.
- Day 3: Implementation of the mobile service dashboard.
- Day 4: Mobile views for document requests and scheduling.
- Day 5: Integration of the mobile service layer with the Spring Boot API.

Week 8: Complete Mobile App

- Day 1: Finalizing mobile document upload and status tracking.
- Day 2: Mobile clinic schedule management for health workers.
- Day 3: UI polish, animations, and ensuring two-click navigation.
- Day 4: Comprehensive testing on physical devices and emulators.
- Day 5: Documentation and final preparation of the mobile build.

Phase 5: Integration & Deployment (Week 9–10)

Week 9: Integration Testing

- Day 1: End-to-end testing of document and health modules across all platforms.
- Day 2: Bug fixes and database query optimization.
- Day 3: Security review of JWT tokens and RBAC enforcement.
- Day 4: Performance testing to ensure 5-second load times.
- Day 5: Final documentation updates for future maintainability.

Week 10: Deployment

- Day 1: Deployment of the backend JAR to a cloud server.
- Day 2: Deployment of the Supabase PostgreSQL database and web application.
- Day 3: Final APK distribution for mobile testing.
- Day 4: Verification of 99% system uptime target.
- Day 5: Final project submission.

Milestones

- M1 (End Week 2): All design models (ERD, Use Case, Activity Diagrams) complete.
- M2 (End Week 4): Backend API with Spring Security fully functional.
- M3 (End Week 6): Web application with responsive dashboards complete.
- M4 (End Week 8): Mobile application fully integrated with the API.
- M5 (End Week 10): Full system integrated with PostgreSQL and deployed.

Critical Path

1. Authentication system: Implementing Spring Security and multi-role registration.
2. Document Transaction API: Developing the core request/approval workflow.
3. Scheduling functionality: Building the clinic calendar and status management.
4. Integration: Ensuring data consistency between web and mobile platforms via Supabase.

Risk Mitigation

- Core-First Approach: Focus on the Resident registration and document request flows before secondary features.
- Continuous Testing: Utilize the ACID properties of JPA to verify transaction persistence daily.
- Modular Codebase: Separate the Auth Module from transaction logic to allow independent security updates.